

Estructuras de Datos PECL

SIMULACIÓN DE TRIAJE Y ATENCIÓN A PACIENTES EN LOS QUIRÓFANOS DE UN HOSPITA

03 / 10 / 2026

Integrantes - Grupo Nº 20:
Nathan Jurado Díaz - **49227414P**
Leonardo Daniel Pérez Camacho - **Y8955006Q**

Profesores del curso:
Carlos Javier Hellín Asensio
Abdelhamid Tayebi Tayebi

La documentación que se entregará junto con el proyecto deberá tener al menos los siguientes
apartados:
a. Nombre y DNI de los alumnos del grupo. Listo
b. Detalles y justificación de la implementación:
b.1 Especificación concreta de la interfaz de los TAD's implementados:
b.1.1 TAD`s creados.
b.1.2 Definición de las operaciones del TAD (Nombre, argumentos y retorno).
b.2 Solución adoptada: descripción de las dificultades encontradas.
b.3 Explicación de los métodos más destacados.
b.4 Explicación del comportamiento del programa.
b.5 Bibliografía.

Borrador de especificaciones (resumen del guion de la práctica)

Pacientes

- DNI con letra.
- Identificador (ID) único: se establece en la fase de preoperatorio en función del tipo de enfermedad.
- Número de habitación (único): se asigna a la vez que el ID.
- Tipo de enfermedad (booleano).
- Todos los datos del paciente se establecen de forma automática.

Preoperatorio — salas

- Dos tipos de salas para cada tipo de enfermos, modeladas con 4 colas dinámicas:
- Salas **A** y **B**: pacientes con apendicitis.
- Salas **C** y **D**: pacientes con hernias.

Preoperatorio — asignación de pacientes a las salas

1. Se generan y almacenan los pacientes en la pila de pacientes.
2. Se irán desapilando para entrar en una de las dos salas posibles (según el tipo de enfermedad):

- Concretamente en la que menos pacientes tenga.

- Ejemplo (apendicitis): sala A = 2 pacientes, sala B = 1 paciente → el paciente se encola en la sala B.
- Si hay el mismo número de pacientes, siempre en la sala A.

Preoperatorio — gravedad / prioridad

- Relacionado con los datos del paciente: a la vez que se genera la enfermedad se genera su gravedad (prioridad): **gravedad = ID del paciente**.
- Gravedad apendicitis (ID paciente): **1–49**.
- Gravedad hernia (ID paciente): **51–99**.
- **Los IDs serán únicos.**

Entrada a quirófano

- Se hará mediante dos listas (una por cada tipo de enfermedad) implementadas dinámicamente y simplemente enlazadas: cada nodo tiene referencia al siguiente elemento.
- Los pacientes se almacenan en la lista de quirófano ordenados por su prioridad.

Postoperatorio

- Se llevará a cabo en habitaciones individuales.
- Operados de apendicitis → primera planta: habitaciones en el rango **101–199**.
- Operados de hernia → segunda planta: habitaciones en el rango **201–299**.

Parte 1

Implementaciones necesarias antes de entrar en la Parte 1:

- Clases **Paciente**, **Cola**, **Pila** y **Lista** (con sus métodos y operaciones).
- Clase **Controlador**: simulará el control de acceso de los pacientes en la clase main, que mostrará el menú:

Opción	Funcionalidad
A	Genera 12 pacientes y los almacena en la pila de pacientes (hasta 48).
B	Mostrar los pacientes almacenados en la pila de pacientes.
C	Borrar todos los pacientes de la pila y los generados antes de entrar en ella.
D	Extrae los pacientes de la pila y los almacena en una de las dos colas según su enfermedad.
E	Muestra los pacientes en las salas A y B (preoperatorio de apendicitis).
F	Muestra los pacientes en las salas C y D (preoperatorio de hernias).
G	Borra los pacientes de las salas del preoperatorio.
H	Extrae los pacientes de las colas del preoperatorio y los almacena en las listas del quirófano según su enfermedad.
I	Muestra los pacientes en la lista de apendicitis.
J	Muestra los pacientes en la lista de hernias.
K	Busca en las listas y muestra los 2 pacientes siguientes: el de apendicitis más grave y el de hernia más grave.
L	Reinicia el programa a su estado original.

Parte 2

Complementa la Parte 1 con la creación de un **árbol binario de búsqueda (ABB)**, implementado dinámicamente, que almacenará los pacientes que están en las habitaciones durante el postoperatorio.

Construcción del ABB

- Primero se crea un NodoArbol con un paciente ficticio con el número de habitación **200**, de manera que:
- Los pacientes con **apendicitis** se insertan ordenados por su número de habitación en el **subárbol izquierdo** del nodo raíz.
- Los pacientes con **hernias** se insertan ordenados por su número de habitación en el **subárbol derecho** del nodo raíz.

Requisitos de implementación

- Se ampliará el menú de la Parte 1 y se añadirá la implementación de las clases NodoArbol y Arbol.
- Para mostrar la información de las opciones del menú, se deberá partir del ABB creado y recorrerlo de la forma más eficiente posible.

Opción	Funcionalidad
M	Crea y dibuja el ABB en consola.
N	Muestra los datos de los pacientes con apendicitis ordenados por número de habitación, de menor a mayor.
O	Muestra los datos de los pacientes con hernias ordenados por su número de habitación.
P	Muestra los datos de todos los pacientes recorriendo el ABB en inorden.
Q	Busca en el ABB estos 4 pacientes: el de apendicitis con nº de habitación menor y mayor, y el de hernia con nº de habitación menor y mayor.
R	Cuenta el número de pacientes almacenados en el ABB cuyos números de habitación son impares.
S	Salir del programa.
T	Muestra los pacientes que se encuentran almacenados en un nodo hoja.
U	Elimina un paciente indicado por su número de habitación (introducido por consola), mostrando el ABB antes y después.

correccion y plantilla para siguiente sesion de lista:

Opción C — Plantilla (template <class T> class Lista)
- Reutilizable para int, Paciente, etc. Lo más "profesional", pero quizá más de lo que pide la práctica.
Cambios que harían falta en cualquier caso:
- NodoLista.hpp: valor pasa de int a Paciente (o Paciente*), y el constructor recibe const Paciente&.
- Lista.hpp: insertarHead/insertarUltimo(const Paciente& v), eliminarHead/eliminarUlitmo devuelven Paciente, verHead/verUltimo devuelven Paciente.
- mostrar(): cout << temporal->valor no compila con un Paciente → hay que llamar a temporal->valor.saludarPaciente() (o añadir operator<< a Paciente).
Antes de tocar nada: ¿quieres que planeemos primero arreglar los 3 métodos rotos de la lista actual, o directamente hacemos la conversión a Paciente? Y si es lo segundo, ¿qué opción prefieres: A (copia), B (puntero) o C (template)?